

Relatório 18 – Docker e Containers para Aplicações

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre Docker e a prática de containerização de aplicações para entender sobre como criar e gerenciar containers, imagens e volumes. A tarefa consistia em ver o curso “Introdução a Docker Containers (*Hands On*)” e desenvolver uma atividade prática para demonstração de compreensão do conteúdo.

2. Desenvolvimento

Explicando primeiramente sobre o conteúdo visto em aula, o Docker é uma plataforma de containerização usada para empacotar uma aplicação com todas as suas dependências em uma unidade isolada e portátil que chamamos de container.

O container, diferente de uma VM que virtualiza o *hardware* completo e roda um sistema operacional próprio, compartilha o kernel do SO *host*, o que acaba deixando mais leve e rápido para inicializar. Essa diferença de arquitetura que deixa o Docker útil em cenários de desenvolvimento, testes e implantação de aplicações, o que deixa que o *software* rode em qualquer ambiente.

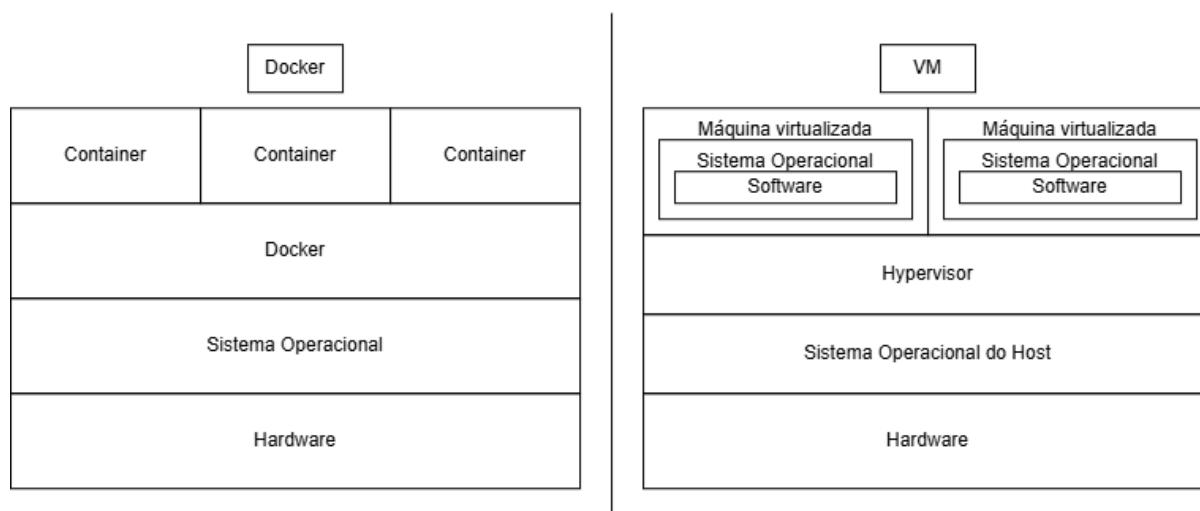


Figura 1. Comparação da Máquina Virtual com o Docker. (autoria própria)

O CLI é a principal forma de interação com o Docker no dia a dia, embora algumas pessoas prefiram usar o Docker Desktop que é a “versão gráfica”. No curso é falado da evolução da sintaxe dos comandos, no formato antigo eram utilizados os comandos usados de forma solta e sem agrupamento por contexto, já no atual são organizados por objeto de gerenciamento com o padrão: `docker <objeto> <ação>`.

Essa organização tornou a CLI mais intuitiva e escalável porque cada objeto passa a ter seu próprio conjunto de subcomandos.

Os containers passam por estados definidos: criado, em execução, pausado e parado/encerrado. O comando *docker run* faz a junção da criação com a inicialização de uma vez só. Se é executado com as *flags -it*, o container é iniciado em modo interativo, que é a alocação de um pseudo-terminal e mantendo o STDIN aberto, desta forma, permitindo ao usuário interagir diretamente com o shell do container (o que é essencial para a depuração e exploração manual do ambiente). Entre os comandos mais utilizados para a manipulação dos containers que já existente são:

- *docker exec -it <container> <comando>*: executa um comando dentro de um container já em execução.
- *docker inspect <container>*: retorna metadados detalhados em formato JSON.
- *docker cp <origem> <destino>*: copia arquivos entre o *host* e o container nas duas direções.
- *docker logs <container>*: mostra a saída padrão do processo principal do container, normalmente é bem importante para saber dos problemas ocorridos sem entrar no container.
- *docker ps*: lista todos os containers em execução. Se adicionar a *flag -a* ele mostra todos os containers.

Os dados que são gerados dentro do container são perdidos quando ele é removido, isso e por padrão, já que o sistema de arquivos do container é meio que de curta duração, mas dá pra resolver isso através do mapeamento de volumes (normalmente pelas *flag -v* ou *-mount*) que resolve o problema vinculando um diretório do *host* a um diretório dentro do container, assim, ele permite a persistência de dados mesmo com a remoção do container. Já para que um serviço rodando dentro seja acessível de forma externa, é possível mapear uma porta do *host* pra uma porta do container, usando a *flag -p <porta_host>:<porta_container>*, isso que vai permitir acessar um servidor *web* dentro de um container pelo navegador do *host*.

Sobre as Imagens, ela é um *template* de apenas leitura, composta de camadas, que serve de base para a criação de containers, pode ser dizer que se um container for um bolo, então a imagem é o kit para fazer o bolo. Enquanto o container é a instância em execução, a imagem é tipo um molde. No curso é mostrado como gerar uma imagem a partir de um container já modificado, usando *docker commit*, ele vai congelar o estado atual do container, assim, gerando uma nova imagem. Para exportar as imagens para arquivos *.tar* é usado o comando *docker save*, o que é bem útil para transporta imagens entre ambientes sem depender de um *registry*, como em ambientes sem acesso à internet. Já o comando *docker load* faz o processo inverso, que importa uma imagem *.tar* para o repositório local de imagens do Docker.

Ainda sobre as imagens, é possível ver as camadas que compõem uma imagem, como os comandos que geraram cada camada e o tamanho que cada uma agrega, o

que é bem útil para entender e otimizar o tamanho final de uma imagem, é usado o comando *docker history <imagem>*. Alguns outros comandos essenciais para o objeto *image*:

- *docker image inspect*: detalhes de uma imagem.
- *docker image prune*: remove imagens não utilizadas, bom para fazer uma limpeza.
- *docker pull*: baixa uma imagem de um *registry* remoto.
- *docker tag*: cria uma tag adicional para uma imagem que já existe.
- *docker push*: envia uma imagem local para um *registry* remoto.

Agora falando do Dockerfile, ele é um arquivo de texto que vai descrever os passos que precisam para construir uma imagem, substituindo o processo manual de criar um container, alterá-lo e depois usar um *commit*. O mais básico inclui *FROM* (imagem base), *RUN* (execução de comandos durante o *build*), *COPY/ADD* (cópia de arquivos para dentro da imagem), *WORKDIR* (diretório de trabalho) e *CMD/ENTRYPOINT* (comando executado ao iniciar o container), o comando *docker build* lê o Dockerfile e gera a imagem final camada por camada.

Prosseguindo para Volumes, no curso é explicado como funcionam por baixo dos panos: volumes gerenciados pelo Docker são armazenados em uma área específica do sistema de arquivo do *host*, gerenciada inteiramente pelo Docker Engine, diferente dos *bind mounts*, em que o usuário aponta diretamente para um caminho arbitrário do *host*.

Finalizando a parte teórica do Docker, agora falando do objeto system, ele reúne comandos de administração de engine como um todo, não de apenas um container ou imagem, alguns comandos interessantes são o *docker system df*, que mostra o espaço em disco ocupado por imagens, containers e volumes, e o *docker system prune*, remove recursos não utilizados como containers parados, imagens sem tag, redes e cache de build, que ajuda a liberar espaço.

Como atividade prática, eu pensei em usar algo que eu já havia feito anteriormente em um projeto pessoal. Neste projeto, eu subi o backend e o banco de dados em um container cada (o frontend foi feito utilizando o Flutter, no dia eu enfrentei um problema de falta de armazenamento e não consegui colocar o frontend em um container, então deixei para fazer em outro momento, no momento, vou mostrar apenas sobre o backend e o banco de dados). Vale constar que o container de banco de dados que eu falei não é um servidor de banco de dados, ele roda uma vez para aplicar as migrations e gerar o cliente do Prisma, depois ele encerra.

O meu sistema é um aplicativo de gerenciamento de tarefas do dia a dia e projetos pessoais, utilizei no backend Fastify + TypeScript, Prisma ORM (SQLite para o banco local, tem algo pronto para usar o PostgreSQL, mas deixei para usar quando eu terminar a versão mobile), Joi, JWT + bcrypt (tava brincando com autenticação e

adicionei até autenticação com uma conta Google, para este projeto não era necessário). Só passei para especificar a parte sobre o que é a aplicação, nos arquivos da atividade prática vou deixar apenas o Dockerfile, mas seguindo para a parte que importa, no Dockerfile foi colocado apenas a instalação das coisas necessárias para a aplicação e expondo a porta 3333 para o acesso remoto, fora do container, e o resto é feito pelo arquivo docker-compose.yml, também criei 3 arquivo .sh para não precisar ficar digitando um comando grande e ser mais rápido para subir, descer ou atualizar o container (start, restart, stop), fiz isso com base no curso sobre o Airflow que também foi apresentado alguns cards atrás.

O container do backend é um servidor Fastify, ele só inicia depois que o container do banco terminar com sucesso. Tanto o backend quanto o banco são construídos a partir do mesmo Dockerfile, ele é baseado em *node:22-bookworm-slim*.

OBS.: Os arquivos enviados serão os que utilizei apenas para o Docker, para replicação é necessário o package.json e package-lock.json do meu projeto pessoal, se quiser o projeto todo, está no meu github, repositório flown. Também será necessário trocar o nome do vários arquivo, só retirar o _pratica.

A seguir, são as prints da tela com a execução dos comandos:

1. Primeiro eu listei todos os containers, fiz uma imagem de backup dos dois containers (eu já os tinha pronto e como eu tinha dados nele, optei por fazer um backup deles agora), apaguei os containers parados e buildei uma nova imagem a partir do Dockerfile subindo dois serviços definidos no compose.

```
Windows PowerShell
PS D:\Projetos\flown> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
03379a0f15ae   flown-backend  "docker-entrypoint.s..."  9 days ago    Up 9 hours   0.0.0.0:3333->3333/tcp, [::]:3333->3333/tcp
132b71162d2c   flown-database "docker-entrypoint.s..."  9 days ago    Exited (0) 9 hours ago
PS D:\Projetos\flown> docker commit 03 backend-backup:v1.0
sha256:90b0b7de44cc7b3e5709cd072fe79fbb3d20154d38a1e67ac22b7540b447e496
PS D:\Projetos\flown> docker commit 13 database-backup:v1.0
sha256:a6b8fa90b4f4608834262840039c2293b4e23c5eb1fa10e4e3e593c3239048eb
PS D:\Projetos\flown> docker compose stop
(-) stop 2/2
Container flown-backend-1 Stopped
Container flown-database-1 Stopped
PS D:\Projetos\flown> docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
03379a0f15ae8eade08d2a31387b3596ec2ac673258a1a2c225fc40111370f8
132b71162d2c4b0720aa7410d06cabeb9b322a476413aeb403c4fa2d5913fef
Total reclaimed space: 639kB
PS D:\Projetos\flown> docker compose -f docker-compose.yml up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 935B 0.0s done
#1 DONE 0.0s
#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 888B 0.1s done
#2 transferring dockerfile: 888B 0.1s done
#2 DONE 0.1s
#3 [backend internal] load metadata for docker.io/library/node:22-bookworm-slim
```

```
Windows PowerShell
#10 [backend 5/6] RUN npm ci
#10 CACHED

#9 [database 4/6] COPY package.json package-lock.json ./
#9 CACHED

#7 [database 2/6] WORKDIR /app
#7 CACHED

#8 [database 3/6] RUN apt-get update && apt-get install -y --no-install-recommends python3 make g++ && rm -rf /var/lib/apt/lists/*
#8 CACHED

#11 [backend 6/6] COPY . .
#11 DONE 0.1s

#11 [database 6/6] COPY . .
#11 DONE 0.1s

#12 [backend] exporting to image
#12 exporting layers 0.1s done
#12 exporting manifest sha256:612db9bfb31f9cf2686b5f6b1328d930301e091569261d895a6bd3674b0ea3 0.0s done
#12 exporting config sha256:e3c6254e639eaa4722ed5bb473376e731731baeacac778105d9b7582adca42 0.0s done
#12 exporting attestation manifest sha256:bd0413e35fa353ee90c34ca3a0d662fabd61afcf7d9ad70a55696953537a7fb 0.1s done
#12 exporting manifest list sha256:0e6104950ef9808f1c39ea4199363644f0e67ae3c1109d9b9fcd29e650c01 0.0s done
#12 naming to docker.io/library/flown-backend:latest
#12 naming to docker.io/library/flown-backend:latest 0.0s done
#12 unpacking to docker.io/library/flown-backend:latest 0.1s done
#12 DONE 0.5s

#13 [database] exporting to image
#13 exporting layers 0.2s done
#13 exporting manifest sha256:b5c92469130cd57a128ea30447a563983ball19668548f598d9744b5efa4a438b 0.0s done
#13 exporting config sha256:6d222e96e45a91d8061edc7bed9b486ee4759fbd7e01ff4950884327566d6fd1 0.0s done
#13 exporting attestation manifest sha256:4a027284f7721c167ebe10e0d5695b01873088f629b08be78840c0d57632e2cf 0.1s done
#13 exporting manifest list sha256:530808f9f4af4d2710777a10774a4e4d4454ef78dc32e32309e3b94ddc25a434 0.0s done
#13 naming to docker.io/library/flown-database:latest
#13 naming to docker.io/library/flown-database:latest 0.0s done
#13 unpacking to docker.io/library/flown-database:latest 0.1s done
#13 DONE 0.6s

#14 [backend] resolving provenance for metadata file
#14 DONE 0.1s

#15 [database] resolving provenance for metadata file
#15 DONE 0.0s

[+] Up 4/4
Image flown-database Built
Image flown-backend Built
Container flown-database-1 Exited
Container flown-backend-1 Started
PS D:\Projetos\flown>
```

Figura 2 e 3. Criando uma imagem nova a partir do container, apagando os containers e a upando via docker compose. (autoria própria)

2. Verificando os containers em execução.

```
Windows PowerShell
PS D:\Projetos\flown> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
flown-backend-1 flown-backend "docker-entrypoint.s..." backend 7 minutes ago Up 7 minutes 0.0.0.0:3333->3333/tcp, [::]:3333->3333/tcp
PS D:\Projetos\flown> docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
1ddc0a9cf17e flown-backend "docker-entrypoint.s..." 7 minutes ago Up 7 minutes 0.0.0.0:3333->3333/tcp, [::]:3333->3333/tcp flown-backend-1
4a6c53b89cf9 flown-database "docker-entrypoint.s..." 15 minutes ago Exited (0) 7 minutes ago 0.0.0.0:3333->3333/tcp, [::]:3333->3333/tcp flown-database-1
```

Figura 4. Listando containers. (autoria própria)

- No comando *docker compose ps* e *docker ps -a* da imagem anterior, pode ver que o *expose 3333* mapeia essa porta para o *host* com “3333:3333” no serviço *flown-backend*, isso é para permitir que o acesso a API rodando dentro do container diretamente no *localhost:3333* a partir do *host*.

- Para ver se o container subiu, fui ver os logs dele.

```
Windows PowerShell
PS D:\Projetos\flown> docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
1ddc0a9cf17e flown-backend "docker-entrypoint.s..." 15 minutes ago Up 14 minutes 0.0.0.0:3333->3333/tcp, [::]:3333->3333/t
4a6c53b89cf9 flown-database "docker-entrypoint.s..." 15 minutes ago Exited (0) 14 minutes ago

PS D:\Projetos\flown> docker logs 1d
{"level":30,"time":1787150168984,"pid":31,"hostname":"1ddc0a9cf17e","msg":"Server listening at http://127.0.0.1:3333"}
{"level":30,"time":1787150168985,"pid":31,"hostname":"1ddc0a9cf17e","msg":"Server listening at http://172.18.0.2:3333"}
{"level":30,"time":1787150170705,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-1","req":{"method":"GET","url":"/notifications?limit=20","host
remotePort":43466},"msg":"incoming request"}
{"level":30,"time":1787150170740,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-1","res":{"statusCode":200},"responseTime":33.751391000000694,
{"level":30,"time":1787150290702,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-2","req":{"method":"GET","url":"/notifications?limit=20","host
remotePort":59040},"msg":"incoming request"}
{"level":30,"time":1787150290712,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-2","res":{"statusCode":200},"responseTime":10.509823999993387,
{"level":30,"time":1787150410685,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-3","req":{"method":"GET","url":"/notifications?limit=20","host
remotePort":39554},"msg":"incoming request"}
{"level":30,"time":1787150410695,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-3","res":{"statusCode":200},"responseTime":9.654051999998046,
{"level":30,"time":1787150530692,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-4","req":{"method":"GET","url":"/notifications?limit=20","host
remotePort":48616},"msg":"incoming request"}
{"level":30,"time":1787150530703,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-4","res":{"statusCode":200},"responseTime":10.3757889999954239,
{"level":30,"time":1787150650704,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-5","req":{"method":"GET","url":"/notifications?limit=20","host
remotePort":44632},"msg":"incoming request"}
{"level":30,"time":1787150650704,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-5","res":{"statusCode":200},"responseTime":9.9190489999960069,
{"level":30,"time":1787150770686,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-6","req":{"method":"GET","url":"/notifications?limit=20","host
remotePort":40920},"msg":"incoming request"}
{"level":30,"time":1787150770693,"pid":31,"hostname":"1ddc0a9cf17e","reqId":"req-6","res":{"statusCode":200},"responseTime":7.2601830000005782,"
```

Figura 5. Conferindo logs. (autoria própria)

- Como deu tudo certo, conferi as imagens, como eu não apaguei as duas última do back e do banco, é para ter 4 imagens.

```
PS D:\Projetos\flown> docker images
back-end-backup:v1.0 90b0b7de44cc 1.54GB 411MB
database-backup:v1.0 a68bfa90b4f4 1.54GB 410MB
flown-backend:latest 0c6104950ef9 1.54GB 410MB U
flown-database:latest 530808f9f4af 1.54GB 410MB U
```

Figura 6. Listando imagens. (autoria própria)

6. Usando o *docker history <imagem>* será possível ver as camadas geradas a partir das instruções do Dockerfile e em seguida irei exportar usando o *save*

```
PS D:\Projetos\flown> docker history 90b0b7de44cc
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
90b0b7de44cc   20 minutes ago  npx tsx watch src/server.ts                  471kB     buildkit.dockerfile.v0
<missing>      9 days ago     CMD ["npx" "tsx" "watch" "src/server.ts"]      0B        buildkit.dockerfile.v0
<missing>      9 days ago     EXPOSE [3333/tcp]                             0B        buildkit.dockerfile.v0
<missing>      9 days ago     COPY . . # buildkit                          537kB     buildkit.dockerfile.v0
<missing>      9 days ago     RUN /bin/sh -c npm ci # buildkit              590MB     buildkit.dockerfile.v0
<missing>      9 days ago     COPY package.json package-lock.json ./ # bui... 139kB     buildkit.dockerfile.v0
<missing>      9 days ago     RUN /bin/sh -c apt-get update && apt-get ins... 293MB     buildkit.dockerfile.v0
<missing>      9 days ago     WORKDIR /app                                  8.19kB     buildkit.dockerfile.v0
<missing>      2 weeks ago    CMD ["node"]                                  0B        buildkit.dockerfile.v0
<missing>      2 weeks ago    ENTRYPOINT ["docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      2 weeks ago    COPY docker-entrypoint.sh /usr/local/bin/ # ... 20.5kB     buildkit.dockerfile.v0
<missing>      2 weeks ago    RUN /bin/sh -c set -ex && savedAptMark="$(... 7.32MB     buildkit.dockerfile.v0
<missing>      2 weeks ago    ENV YARN_VERSION=1.22.22                      0B        buildkit.dockerfile.v0
<missing>      2 weeks ago    RUN /bin/sh -c ARCH= OPENSSL_ARCH= && dpkgAr... 154MB     buildkit.dockerfile.v0
<missing>      2 weeks ago    ENV NODE_VERSION=22.23.2                     0B        buildkit.dockerfile.v0
<missing>      2 weeks ago    RUN /bin/sh -c groupadd --gid 1000 node &&... 69.6kB     buildkit.dockerfile.v0
<missing>      2 weeks ago    # debian.sh --arch 'amd64' out/ 'bookworm' '...' 85.3MB     debuerreotype 0.17

PS D:\Projetos\flown> docker save -o flown-backend.tar 90b0b7de44cc
PS D:\Projetos\flown> ls

Diretório: D:\Projetos\flown

Mode                LastWriteTime         Length Name
----                -
d-----          18/08/2026    01:15         .claude
d-----          25/07/2026    16:43         .vscode
d-----          19/08/2026    08:46         backend
d-----          19/08/2026    01:56         build
d-----          11/08/2026    20:58         docs
d-----          10/08/2026    02:52         frontend
d-----          20/07/2026    13:08         log
-a-----          16/07/2026    00:15           530 .env
-a-----          15/07/2026    23:54           66 .gitattributes
-a-----          10/08/2026    01:23          2528 .gitignore
-a-----          10/08/2026    01:23         1001 docker-compose.yml
-a-----          19/08/2026    11:53         410558976 flown-backend.tar
-a-----          26/07/2026    23:17          1095 LICENSE
-a-----          27/07/2026    00:22           911 README.md
-a-----          10/08/2026    01:23           106 restart.sh
-a-----          10/08/2026    01:23           172 start.sh
-a-----          10/08/2026    01:23           56 stop.sh
```

Figura 7. Camadas de instruções do Dockerfile e salvando uma imagem via arquivo .tar. (autoria própria)

7. Só apaguei as imagens que havia criado antes (aliás, havia errado o comando ainda).

```
PS D:\Projetos\flown> docker images

IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
backend-backup:v1.0  90b0b7de44cc      1.54GB      411MB
database-backup:v1.0  a68bfa90b4f4      1.54GB      410MB
flown-backend:latest  0c6104950ef9      1.54GB      410MB  U
flown-database:latest  530808f9f4af      1.54GB      410MB  U

PS D:\Projetos\flown> docker rm backend-backup:v1.0
Error response from daemon: No such container: backend-backup:v1.0
PS D:\Projetos\flown> docker image rm backend-backup:v1.0
Untagged: backend-backup:v1.0
Deleted: sha256:90b0b7de44cc7b3e5709cd072fe79fbb3d20154d38a1e67ac22b7540b447e4
PS D:\Projetos\flown> docker images

IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
database-backup:v1.0  a68bfa90b4f4      1.54GB      410MB
flown-backend:latest  0c6104950ef9      1.54GB      410MB  U
flown-database:latest  530808f9f4af      1.54GB      410MB  U

PS D:\Projetos\flown> docker image rm database-backup:v1.0
Untagged: database-backup:v1.0
Deleted: sha256:a68bfa90b4f4608834262840039c2293b4e23c5eb1fa10e4e36593c3239048
PS D:\Projetos\flown> docker images

IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
flown-backend:latest  0c6104950ef9      1.54GB      410MB  U
flown-database:latest  530808f9f4af      1.54GB      410MB  U
PS D:\Projetos\flown>
```

Figura 8. Apagando as imagens. (autoria própria)

3. Conclusão

O curso ajudou me ajudou em várias coisas, até então eu havia bem pouco conhecimento sobre o Docker e como usar ele, a instalação dele eu não falei sobre, mas é só seguir os passos da documentação dele e conceitos mais avançados como a construção de imagens via Dockerfile, acredito eu, foi uma que eu não tinha uma certa ideia de como fazia. O entendimento sobre a diferença entre imagem e container, funcionamento de volumes e do mapeamento de portas é algo que é essencial para qualquer projeto real, também para não ter o famoso “na minha máquina funciona”.

4. Referências

Docker Docs, <https://docs.docker.com/>

FONSECA, Adriano da Silva, 2021. “Introdução a Docker Containers (Hands On)”. disponível em: <https://www.udemy.com/course/docker-para-devops-atualizado-docker-ce-1706/?couponCode=CP260817G1>.